Journal of Intelligent & Robotic Systems (2025) 111:81
https://doi.org/10.1007/s10846-025-02287-4

REGULAR PAPER

CF-HMRTA: Coalition Formation for Heterogeneous Multi-Robot Task Allocation

Ashish Verma[1] · Avinash Gautam[1] · Ayan Dutta[2] · Virendra Singh Shekhawat[1] · Sudeept Mohan[1]

Received: 11 February 2025 / Accepted: 25 June 2025 / Published online: 23 July 2025
© The Author(s) 2025

Abstract

This paper introduces a novel approach, *Coalition Formation for Heterogeneous Multi-Robot Task Allocation* (CF-HMRTA), to address the challenge of multi-robot task allocation. The problem, inherently NP-Hard, is tackled using bipartite graph matching. CF-HMRTA forms heterogeneous robot coalitions with unique service skills to complete tasks collaboratively, using a heuristic algorithm for optimal robot-task pairing while preventing task overlap. Recent research work using bipartite graph matching for multi-robot coalition formation and task allocation often assumes homogeneity across tasks and robots, where any robot can be assigned to any task. In contrast, the solution proposed in this paper explicitly considers the diversity of robots with varying service skills. Additionally, tasks demand different sets of skills, such as sensing, monitoring, and data collection, making certain tasks unsuitable for some robots due to hardware constraints. For instance, tasks requiring aerial footage are assigned to drones, while ground robots handle close-ground monitoring. Furthermore, we incorporate task-specific time constraints into our problem formulation, enhancing its realism. Considerably less research has been conducted on heterogeneous robot teams solving tasks that require multiple service skills and temporal constraints, making our work a significant contribution to the field. The algorithm achieves a worst-case time complexity of $O(|E|)$, where E represents the edges in the bipartite graph, and guarantees perfect matching. Simulation results highlight its scalability, successfully allocating up to 2000 robots to 400 tasks in approximately 11 seconds.

Keywords Task allocation · Coalition formation · Multi-robot systems

1 Introduction

In many real-world applications, we can use bipartite graph matching to match robots to tasks. This graph-theoretic technique is used in various activities such as classroom scheduling [1], hot desking in shared workspaces [2], customer-taxi pairing [3], and assignment of reviewers to papers [4]. Bipartite matching typically involves pairing elements from two distinct sets, A and B, in a manner that ensures an element in set A is matched uniquely with an element in set B, and vice versa. If all the nodes from either A or B are matched, it is a *perfect matching*. In **M**ulti-**R**obot **T**ask **A**llocation (MRTA), the above-stated property is especially relevant in situations where the aim is to establish exclusive relationships between the elements of the two sets, i.e., a set comprising of tasks that are required to be completed and the other is the set of robots deployed for executing these tasks. Matching using a bipartite graph ensures that a particular task is not assigned to multiple robots, aligning with the **S**ingle **T**ask-**S**ingle **R**obot **T**ime-extended **A**ssignment (ST-SR-TA) problem [5]. Time-extended assignment means the assignment of tasks to the robots based on already available information, i.e., available tasks and robots [6]. The assumption in this setting is that one robot is enough to complete a given task.

✉ Ashish Verma
p20200020@pilani.bits-pilani.ac.in

Avinash Gautam
avinash@pilani.bits-pilani.ac.in

Ayan Dutta
a.dutta@unf.edu

Virendra Singh Shekhawat
vsshekhawat@pilani.bits-pilani.ac.in

Sudeept Mohan
sudeeptm@pilani.bits-pilani.ac.in

1 Department of Computer Science and Information Systems, Birla Institute of Technology and Science, Birla Institute of Technology and Science, Pilani 333031, Rajasthan, India

2 School of Computing, University of North Florida, Jacksonville 32224, Florida, USA

In [7], the authors introduce a novel variant of bipartite matching, named *one-to-many bipartite matching* (OTMaM). Their proposed OTMaM algorithm is specifically designed to handle scenarios in Multi-Robot Systems where one robot might not be enough to complete a task and multiple robots might need to collaborate for that. More specifically, they focus on the **S**ingle-**T**ask **M**ulti-**R**obot **I**nstantaneous **A**ssignment (ST-MR-IA) problem. In this setting, each robot can only perform one task at a time, and multiple robots might be needed to complete any task. Instantaneous allocation means that the allocation process does not involve planning for future assignments [8]. The authors in [7] also assume homogeneity across tasks and robots, i.e., any robot can be assigned to any task. In [9], the authors expanded on [7] by introducing heterogeneity in both robots and tasks. In this scenario, each task might require multiple robots with different sensors/actuators. They used a hedonic game-theoretic formulation for coalition (team) formation among the robots. Their algorithm could scale up to 2000 robots and 400 tasks [9]. Unlike [9], which primarily addresses heterogeneity in capabilities via hedonic coalition formation, CF-HMRTA explicitly incorporates both skill diversity and task-specific deadlines into the optimization objective, minimizing the overall penalty due to deadline violations.

In contrast to the methodology in [7], our approach considers the diversity of robots with varying service skills. Additionally, the execution of tasks demands different sets of skills, such as sensing, monitoring, data collection, etc. This means that certain tasks may not be suitable for certain robots due to hardware constraints. For instance, tasks needing aerial footage are assigned to drones, while ground robots handle tasks like close ground monitoring. Additionally, we consider time constraints for each task in our problem formulation to make it more realistic. This feature is missing in [7, 9, 10].

To this end, we employ a novel allocation technique that leverages the National Resident Matching Program (NRMP) model used in the medical field [11]. Similar to NRMP's process of creating Rank Order Lists (ROLs), our formulation involves robots and tasks submitting preferences. Next, we model this as an ST-MR-IA problem instance ensuring efficient task completion while maximizing robot resource utilization. This adaptation promotes fairness and transparency, aligning both robot and task preferences to optimize system performance. The main contributions of this paper are as follows:

- In this paper, a novel task allocation algorithm i.e., Coalition Formation for Heterogeneous Multi-Robot Task Allocation (CF- HMRTA) is proposed for ST-MR-IA for a team of heterogeneous mobile robots and tasks with time constraints. To the best of our knowledge, this is the first attempt to address the coalition formation problem for heterogeneous robot task allocation using the National Resident Matching Program formulation.
- The proposed algorithm achieves linear time complexity relative to the number of edges in a bipartite graph. The proposed algorithm also yields near-optimal outcomes almost always.

The following sections of this paper are structured as follows: Section 2 reviews related work. Section 3 defines the problem and introduces relevant variables and their notations. Next, we describe the proposed algorithm in Section 4. Section 5 discusses the simulation environment configuration and presents the numerical results. Finally, the conclusion and future works are discuss in Section 6.

2 Related Work

Task allocation problems in multi-robot systems have been extensively studied in prior research, including studies such as [5, 12–15], among others. These papers mainly focus on robot requirements, task allocation algorithm and analysis, and allocation effectiveness evaluation [16]. Initial exploration of coalitions in MRS was conducted in [17]. The authors proposed a greedy algorithm aiming for solutions within a factor of $(k+1)$ of the optimal, where k is the maximum coalition size. In [18], authors propose a modified algorithm with complexity $O(n^{3/2}m)$, compared to $O(n^k m)$ in [17], where n, m is the number of robots and tasks, respectively. Coalition formation algorithm categorization and taxonomy were introduced in [5, 19]. The problem has been identified as NP-Hard [5, 17], which has motivated the researchers to find scalable approximation solutions.

In [5], it was noted that the ST-SR-TA or ST-SR-IA variant can be restated as a bipartite matching problem. Bipartite matching, extensively studied in graph theory, has seen numerous theoretical advancements, with several algorithms proposed in the literature, as noted in [7, 9, 20] and [21]. Algorithms like the Hungarian method [21] have found application in ST-SR-TA assignments [22], aiding in generating non-interfering paths for robots efficiently. In [7], the focus is on weighted matching problems while using it for the ST-MR-IA variant, where each edge carries a weight (representing the cost) and the goal is to minimize the total weight of matched edges. Tasks are allocated to robots greedily based on the distance cost metric, and this is achieved in linear time on the number of edges in the bipartite graph. Inter-task resource constraints for ST-MR-TA assignments are considered in [23].

In [24, 25], the authors explored the allocation problem with time constraints, where robots can perform only one task at a time, and each task requires only one robot. On the other hand, [26] considers ST-MR-TA with task

deadlines. The authors in [27] have solved the ST-SR-TA problem with deadlines where any robot can be assigned to any task, and all tasks take the same time to complete. In contrast to [27], authors in [28] introduce a novel algorithm derived from the GRAPE algorithm, where both tasks and robots are heterogeneous, and tasks may require multiple skills. The time complexity of the algorithm proposed in [28] is $O(n^3 d_G)$, where n represents the number of agents and d_G denotes the diameter of the communication network topology. This algorithm focuses exclusively on coalition formation without considering task deadlines. In contrast to [28], which presents a GRAPE-based coalition formation approach without accounting for task deadlines, CF-HMRTA incorporates an NRMP-inspired ranking mechanism that captures mutual compatibility preferences between robots and tasks while explicitly considering task deadlines. Furthermore, our method represents the robot-task matching as a bipartite graph and introduces a coalition restructuring phase (refer to Algorithm 3), enabling post-assignment refinement. This additional optimization step is not included in [28].

To the best of our knowledge, there is no existing research that directly addresses the unique challenges of heterogeneous multi-robot task allocation under time constraints using bipartite matching. Unlike previous works, we consider that each robot possesses a set of service skills. Thus, a particular robot can be assigned only to a subset of tasks. Moreover, to accomplish a task more than one robot is required, and unlike [27], all the tasks take different amounts of time to complete. In our scenario, delays in accomplishing tasks incur penalties, which presents a novel aspect to consider. Our main aim is to efficiently assign tasks to robots, reducing the cumulative penalty for the robot team while keeping the algorithm's time complexity minimal.

3 Problem Setup

Robots The set of n heterogeneous robots is denoted by $R = \{r_1, \cdots, r_n\}$. Each robot r_i has three attributes encapsulated in a triplet $< L_{r_i}, v_{r_i}, SS_{r_i} >$, where L_{r_i} is the location of the robot, v_{r_i} is the speed of the robot, and SS_{r_i} denotes the set of unique service skills that r_i can accomplish, like monitoring, sensing, etc. $SS_{r_i} = \{SS_1, \cdots, SS_p\}$. $S = \bigcup_{i=1}^{n} SS_{r_i}$, where S denotes the set of all possible service skills the robot team possesses. The set of service skills for the two robots may be identical. We assume that p is a finite value, where $p \geq 2$. The robots can move at different speeds. The robots capable of moving faster are usually dispatched to accomplish tasks with shorter deadlines. Finally, each robot can localize itself in the environment.

Tasks The set of m tasks is represented as $T = \{t_1, \cdots, t_m\}$. Each task t_j has five attributes encapsulated within the tuple $< L_{t_j}, RS_{t_j}, \delta t_j, D_{t_j}, N_{t_j} >$, where L_{t_j} denotes the location of the task, RS_{t_j} denotes a set of required skills, δt_j is the duration to execute the task, D_{t_j} represents the deadline of the task, and N_{t_j} is the number of robots required to accomplish the task, which is equal to $|RS_{t_j}|$. $RS_{t_j} = \{RS_1, \cdots, RS_q\}$. $S = \bigcup_{j=1}^{m} RS_{t_j}$. $RS_{t_j} \in S$. Tasks may potentially have the same set of required skills. We assume that q is a finite value, where $q \geq 2$, and $n \geq 2m$.

In our framework, a central planner is responsible for assigning tasks to robots. Following prior work such as [7, 29], we assume a reliable communication infrastructure-a reasonable assumption in controlled environments such as warehouses, office buildings, and hospitals. Accordingly, CF-HMRTA operates under the assumption of stable communication across the system, where each robot maintains a continuous connection with the central planner. This ensures timely delivery of task assignments and updates, thereby preventing delays or conflicts during execution and supporting accurate, efficient task distribution. By adopting this assumption, we focus on evaluating the core task allocation strategy without the added complexity introduced by communication failures.

Feasibility of Robots and Task Matching Before task allocation, it is essential to establish the pairing between robots and tasks, denoted as $M = \{(r_i, t_j) \mid r_i \in R, t_j \in T\}$. The set M consists of all possible pairs of robots and tasks. The utility of the robots to execute the tasks is defined as $U(r_i, t_j) = SS_{r_i} \cap RS_{t_j}$, which represents the INTERSECTION operation between service skill r_i and required skill of t_j [30]. If $|U(r_i, t_j)| \geq 1$, then it means the robot r_i *can be paired* with task t_j, otherwise $|U(r_i, t_j)| = 0$.

Coalition Formation Since each task necessitates the collaboration of multiple robots, a coalition, c, is established as a proper subset of the robot set R, denoted as $c \subseteq R$. Each robot within a coalition c is required to work collectively to execute the assigned task. A coalition structure, i.e., the initial assignment solution, is denoted as $CF = \{c_1, \cdots, c_m\}$, $c_i \cap c_j = \phi, \forall c_i, c_j \in CF$ and $i \neq j$. The coalition structure CF represents a collection of non-overlapping groups of robots that collectively cover R. Here, c_j denotes the coalition of robots allocated for task t_j, where $j = \{1, \cdots, m\}$.

Cost Function In the beginning, the robots are placed randomly within the environment. Upon allocation to a task as part of a coalition, a robot must autonomously navigate towards the task location L_{t_j}. Therefore, the cost of pairing a robot and a task is defined as:

$$cost(r_i, t_j) = \frac{dist(L_{t_j}, L_{r_i})}{v_{r_i}} \tag{1}$$

Here, $dist(L_{t_j}, L_{r_i})$ represents the distance between the locations of robots and tasks denoted by L_{r_i} and L_{t_j} respectively.

Upon reaching the task location L_{t_j}, each robot must invest a certain amount of time, δt_j, to execute the task. The cost of the coalition is the sum of the maximum cost incurred by the farthest robot (defined in Eq. 1) and the duration to execute the task (δt_j). Formally, it is defined as:

$$\omega_{t_j} = \max_{r_i \in c_j} cost(r_i, t_j) + \delta t_j \tag{2}$$

where ω_{t_j} represents the overall working time to execute the task t_j. ω_{t_j} is composed of two parts: the first term, $\max_{r_i \in c_j} \text{cost}(r_i, t_j)$, captures the maximum travel time required by any robot r_i within the coalition c_j assigned to the task. The second term, δ_{t_j}, denotes the time needed to execute or complete the task t_j after all the robots have arrived.

Penalty Function When a task t_j is assigned to a coalition of robots c_j, and if all the robots execute the assigned task (defined in Eq. 2) before its respective deadline (D_{t_j}), then no penalty is incurred. However, if the robots finish the task after the deadline, a penalty is imposed. The penalty function is defined as:

$$pen(c_j) = \max(0, \omega_{t_j} - D_{t_j}) \tag{3}$$

Here, $pen(c_j)$ represents the incurred penalty for the coalition of robot (c_j) to execute the task (t_j) (defined in Eq. 3). The cost of executing all tasks with or without penalty by the coalitions of robots is defined as:

$$penalty(CF) = \sum_{j=1}^{m} pen(c_j) \tag{4}$$

Objective The goal of the examined variant of the ST-MR-IA problem is to determine m non-overlapping coalitions for m tasks, ensuring that the resulting coalition structure (CF^*) minimizes the penalty while assigning N_{t_j} robots to each task t_j.

$$CF^* = \arg\min_{CF} penalty(CF), \text{ s.t., } |c_j| = N_{t_j}, \forall j \in \{1, \cdots, m\} \tag{5}$$

Upon receiving the initial allocation through CF, each robot will assess the potential benefits of collaborating with other robots in different coalitions. This evaluation involves considering various factors such as the nature of the tasks involved, the capabilities of the other robots, and the overall incurred penalty (refer to Eq. 4) that can be further minimized from joining a particular coalition (refer to Eq. 5).

4 Methodology

This section elaborates on the significance of the National Resident Matching Program (NRMP) and weighted maximum bipartite matching in ST-MR-IA. Following that is the detailed explanation of the proposed algorithm, called CF-HMRTA .

4.1 National Resident Matching Program (NRMP)

The National Resident Matching Program (NRMP) algorithm pairs medical graduates with residency programs by optimizing mutual preferences, ensuring that both entities are satisfied with their matches [31]. This algorithm handles complex preferences and guarantees stability, meaning no two participants would prefer a different match over their current one. The problem of Coalition Formation for Heterogeneous Multi-Robot Task Allocation (CF- HMRTA), introduced in this paper, shares several key characteristics with the NRMP. In CF- HMRTA , robots with varying skills are assigned to tasks that may require cooperation, akin to how medical students and residency programs have mutual compatibility based on specific preferences. The NRMP's preference-based matching ensures that tasks are allocated to the most suitable robots, enhancing overall efficiency and satisfaction. Moreover, the NRMP algorithm's ability to ensure stable matches is analogous to forming stable coalitions of robots and tasks in CF- HMRTA , preventing any pair from preferring an alternative match that could destabilize the current allocation. The NRMP is adept at handling complex matching scenarios, which is essential in CF- HMRTA , where tasks may have intricate requirements met by coalitions of robots, ensuring all necessary task requirements are fulfilled. Additionally, the NRMP's focus on optimizing the overall matching outcome translates in CF- HMRTA to minimizing penalties in task allocation, which is crucial for time-sensitive tasks that require efficient use of robot skill sets. Finally, the computational efficiency of the NRMP makes it well-suited for CF- HMRTA , where scalable and feasible task allocation processes involving many robots and tasks are essential.

Algorithm 1 states the functionality of the NRMP. Initially, it takes all the participants as unmatched (Line 1). Residents apply to programs based on their preferences, and programs tentatively accept or reject based on their own preferences (Line 3 - Line 21). The algorithm iteratively adjusts matches by allowing residents to apply to their next preference if rejected, and programs update their matchings by

Algorithm 1 NRMP algorithm.

1: **Input:** A list of medical residents, each with a preference list of residency programs, a list of residency programs, and the number of available slots for each residency program.
2: **Output:** A stable matching of medical residents to residency programs.
3: **while** there is a resident who is unmatched and has not yet applied to every program on their preference list **do**
4: Select an unmatched resident R
5: R applies to the highest-ranked program P on their preference list that they have not yet applied to
6: **if** P has an open position **then**
7: P temporarily accepts R
8: R is marked as matched
9: **else**
10: P compares R with the lowest-ranked resident R' currently accepted
11: **if** R is ranked higher than R' on P's preference list **then**
12: P rejects R'
13: P temporarily accepts R
14: R is marked as matched
15: R' becomes unmatched
16: **else**
17: P rejects R
18: R remains unmatched
19: **end if**
20: **end if**
21: **end while**
22: Continue the process until all residents are either matched or have applied to every program on their list
23: The algorithm terminates when no further rejections are possible
24: The final match is the set of all resident-program pairs where residents are matched to programs

comparing new applicants with current matches (Line 22). The algorithm terminates when no resident can improve their situation, leading to a stable match (Line 23). In the end, the algorithm returns the set of all resident-program pairs where residents are matched to programs (Line 24).

4.2 Preference-Based Assignment Strategy

Algorithms 1 and 2 both initiate by defining the problem context - whether it involves matching residents to programs or assigning robots to tasks. Despite operating in different domains, both algorithms follow a similar structural logic aimed at achieving stable and mutually agreeable matches. In the case of NRMP (Algorithm 1), participants' preferences are explicitly defined prior to execution (Line 5), reflecting subjective priorities based on predefined criteria. Conversely, Algorithm 2 calculates the feasibility between robots and tasks (Line 12), enabling a data-driven preference ranking. Following the initialization of preferences or feasibility scores, both algorithms enter an iterative matching process. In each iteration, entities (residents or robots) apply to their most preferred available option, while the receiving side evaluates incoming proposals based on their own preferences or compatibility. This process leads to provisional assignments that are subject to change as better options emerge. If a proposed match offers a higher preference or improved feasibility compared to the current assignment, it is accepted, and the previous match is dissolved. This iterative refinement continues until a stable state is reached–defined as a configuration in which no entity would prefer to be matched differently without making another participant worse off. A key distinction is that Algorithm 2 incorporates a more flexible matching process tailored for real-time multi-robot coordination, where feasibility can evolve due to task requirements. Moreover, in the following subsection, the algorithm includes mechanisms such as coalition restructuring (Algorithm 3) that allow further optimization after the initial matching is completed–an enhancement not present in the standard NRMP framework.

4.3 The Proposed Approach

An undirected, weighted bipartite graph is defined as $G = (R, T, E)$, where R and T are vertex sets representing robots and tasks, respectively. The edge set E consists of all possible connections between pairs $\langle r, t \rangle$, with $r \in R$ and $t \in T$. A bipartite graph is constructed with tasks and robots as vertices. Each edge of the bipartite graph is formed based on the feasibility of matching between robots and tasks as defined in the previous section (refer to Section 3).

Algorithm 2 takes as input a bipartite graph G. Also, the following variables are initialized.

- `completed_tasks`: Set of tasks which have been assigned the required number of robots (Line 3). Initially, the set is empty.
- `matching`: A data structure to store the assignments of robots to tasks according to robot-task compatibility (Line 4).
- M: A data structure to store task Id t_j as the key and the set of required skills for that task as the value. For instance, task t_1 would be the key and its set of required skills $RS_{t1} = \{RS_1, RS_2, RS_3\}$ as the value.
- A: A data structure to store robot Id r_i as the key and the number of service skills of the robot as the value. For instance, a robot r_1 would be the key and its associated set of service skills $SS_{r1} = \{SS_1, SS_2, SS_3\}$ as the value.
- K: It is a data structure that stores service skill (SS_i) as the key and the robot Ids that match with the SS_i as the value. For instance, a task t_1 would be the key and its associated set of required skills $SS_1 = \{r_1, r_2, r_3\}$ as the value.

The algorithm iterates through each task; therefore, none of the tasks are unattended. M' is a set of required skills of the task. For each required skill of the task, find the set of robot

Algorithm 2 NRMP based multi-robot task assignment (CF-NRMP).

1: **Input:** A graph $G = (R, T, E)$, M, A, K
2: **Output:** CF: A tuple containing the allocation of robots to tasks (matching)
3: `completed_tasks` = { }
4: `matching` = { }
5: **for** $i = 1 \ldots m$ **do**
6: $M' = M.get(t_i)$
7: **for** each $j \in M'$ **do**
8: `robot_assigned` = null
9: `max_ratio` = 0
10: $K' = K.get(j)$
11: **for** each $k \in K'$ **do**
12: $ratio = \frac{N_{t_i}}{A.get(k)}$
13: **if** $ratio \geq$ `max_ratio` **then**
14: `max_ratio` = $ratio$
15: `robot_assigned` = k
16: **end if**
17: **end for**
18: `matching`[t_i] = `matching`[t_i] ∪{`robot_assigned`}
19: $M.get(i) = M.get(i) \setminus j$
20: $K.get(j) = K.get(j) \setminus$ {`robot_assigned`}
21: $N_{t_i} = N_{t_i} - 1$
22: **end for**
23: **if** $N_{t_i} = 0$ **then**
24: `completed_tasks` = `completed_tasks` $\cup t_i$
25: $T = T \setminus t_i$
26: **if** size of `completed_tasks` = m **then**
27: **return** `matching`
28: **end if**
29: **end if**
30: **end for**
31: **return** `matching`

id that is compatible with the required skill, i.e., K'. For each robot belonging to the set K', find the *ratio*, i.e., calculated between the number of robots required for a task (N_{t_i}) and the total number of robots capable of performing that task ($A.get(k)$) (Line 12). The ratio will select the robot which has the least number of service skills. The selected robot for the task is stored in the data structure `matching`. After assigning a task to a robot, the algorithm updates `matching` (Line 18) and other data structures as below.

- It removes the required skill from the task, i.e., already assigned to the robot (Line 19).
- It removes the assigned robot from the K (Line 20).
- It decrements the number of available robots for the task (Line 21).
- If the task gets the required number of robots, it updates the `completed_tasks` set (Line 23).

The algorithm then checks if the number of tasks equals the number of completed tasks. If true, it returns the `matching` (Line 26 - Line 27); otherwise, continue the loop. Finally, the algorithm returns the `matching`, which contains the assignments of robots to tasks (Line 31). The algorithm will terminate when all the tasks get the required number of robots, and all the robots are assigned to the tasks based on their compatibility.

Algorithm 3 Coalition formation for heterogeneous multi-robot task allocation (CF-HMRTA).

1: **Input:** A graph $G = (R, T, E)$, attributes of each task $t_j \in T$
2: **Output:** Penalty after matching and assigned tasks
3: $CF^* \leftarrow$ Algorithm 2
4: pruned $G \leftarrow G - CF^*$
5: $Z \leftarrow$ True
6: $I \leftarrow 1$ ▷ I is a variable counting the number of iterations of the loop
7: **while** Z **do**
8: Go through t_i in CF^* in non-increasing order of penalty
9: $I \leftarrow I + 1$
10: Find the farthest robot r_i connected to t_i in CF^*
11: Find the RS_a of t_i for which r_i is assigned in CF^*
12: Go through each SS_b in pruned G sharing an edge with r_i
13: Go through each t_j having the required skill RS_b in G except t_i
14: Find the robot r_j assigned to the RS_b in t_j in CF^*
15: **if** ($SS_a \in$ service skills of r_j) and ($SS_b \in$ service skills of r_i) **then**
16: **if** $cost(r_i, t_j) < cost(r_i, t_i)$ and $cost(r_j, t_i) < cost(r_j, t_j)$ **then**
17: Update CF^*
18: **end if**
19: **end if**
20: **if** $I == m$ **then**
21: $Z \leftarrow$ False
22: **end if**
23: **end while**
24: Broadcast CF^* to every robot
25: Move to the allocated task locations as per CF^*
26: $P \leftarrow$ calculate penalty of CF^* using Eq. 4
27: **return** P

After getting the initial coalition CF from Algorithms 2, 3 proceeds with the overall task allocation method while further minimizing the overall penalty. It takes an undirected, weighted bipartite graph and returns the overall incurred penalty Eq. 5 while updating the formed CF^* as necessary. The deadline duration of each task is generated following [32]. After forming the coalitions for the tasks, first, it prunes the graph G by removing the edges that are already matched as per the obtained CF (Line 4). Algorithm 3 iterates through tasks t_i, in CF in descending order of penalty (Line 8). The algorithm checks if it is possible to swap two tasks, t_i and t_j, between robots, r_i and r_j, respectively, while maintaining the feasibility of robot-task matching and reducing the penalty (Line 15). If the swapping of robots reduces the penalty, update the coalition formation accordingly (Line 17). As outlined in the pseudocode (Line 20 to Line 22), the process is bounded by at most m iterations, where m denotes the number of tasks, with a finite number of swap opportunities evaluated in each iteration. Although we do not claim global optimality or convergence guarantees, the algorithm is guaranteed

to terminate within m iterations, ensuring a polynomial runtime. This worst-case behavior is explicitly stated in Section 4.4. A leader robot broadcasts the updated coalition formation to all robots (Line 24). Next, the robots move to their assigned task locations based on the updated coalition information (Line 25). The overall penalty incurred is calculated as per Eq. 4 (Line 26).

4.4 Time-complexity Analysis

In Algorithm 2, the main loop iterates over the number of tasks, denoted as m (Line 5). Consequently, the worst-case time complexity of the main loop is $O(m)$. Within this main loop, there are nested loops. The first inner loop (Line 7 to Line 22) iterates over the number of required skills for each task, represented as $(|RS_{t_j}|)$, which occurs p times and takes $O(p)$ time. The second inner loop (Line 11 to Line 17) iterates over the number of robots that possess the service skills compatible with each required skill of t_j, occurring at most n times and taking $O(n)$ time. Thus, the overall time complexity of the algorithm is $O(mnp)$. Given that p is constant for a given application, the complexity simplifies to $O(mn)$.

For Algorithm 3, the complexity is determined by the number of times the loop runs (Line 7), which is at most m times. Identifying the farthest robot r_i for the task t_j (Line 10) assigned for the required skill RS_m (Line 11) takes constant time, making this step constant in complexity. A robot can have a maximum of p service skills, so the complexity for Line 12 is at most $O(p)$. Since an entity of required skill can be associated with all tasks (Line 13), this step can run up to m times, resulting in a time complexity of $O(m)$. For Line 14, the time complexity remains constant. Therefore, the worst-case time complexity of Algorithm 3 is $O(m \times m)$, or $O(m^2)$.

Consequently, the overall worst-case time complexity of the proposed algorithm is $O(mn) + O(m^2)$, which simplifies to $O(mn)$ since n is greater than m. In the case of a complete bipartite graph G, where $|E| = mn$, the complexity can also be expressed as $O(|E|)$, as referenced in [20].

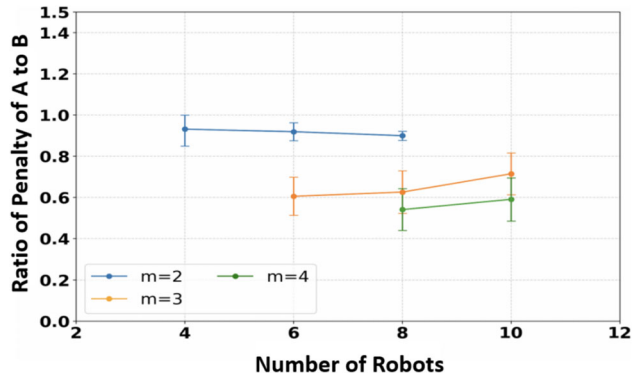

Fig. 1 Ratio of penalty where **A** is the optimal and **B** is the proposed approach: **CASE-1**

5 Experiments and Results

The proposed approach for heterogeneous multi-robot online task allocation has been rigorously evaluated through extensive simulation experiments. This section describes the simulation setup and presents the results. To ensure a robust evaluation, robot task matching based on NRMP methods (CF-NRMP) was re-implemented and thoroughly tested under the same conditions. The proposed approach is directly compared with CF-NRMP, and the results of this comparison are discussed in detail. Furthermore, the coalition switching method (refer to Algorithm 3), a key feature of the proposed approach, was integrated into Algorithm 2 to evaluate its impact. The results of coalition switching are analyzed, showcasing its effectiveness in enhancing the overall efficiency of task allocation among heterogeneous robots. These findings underscore the advantages of the proposed approach.

Table 1 Deadline configuration cases

Case	Deadline Configuration	Characteristics	Representative Applications
CASE-1	$D = E$	• Tight deadlines • High likelihood of task penalties • Worst-case stress test	Emergency response, hazardous event monitoring
CASE-2	$D = 2E$	• Moderate deadlines • Some task delay is tolerable • Intermediate difficulty	Campus delivery bots or indoor logistics
CASE-3	$D \sim U(5E, 10E)$	• Generous deadlines • Most tasks completed on time • Favourable conditions	Infrastructure inspection, agriculture, warehouse tasks
CASE-4	$D \sim U(E, 10E)$	• Mixed range • No bias to tight or relaxed timing • Balanced, general-purpose scenario	Multi-mission autonomous deployments, robotic fleets

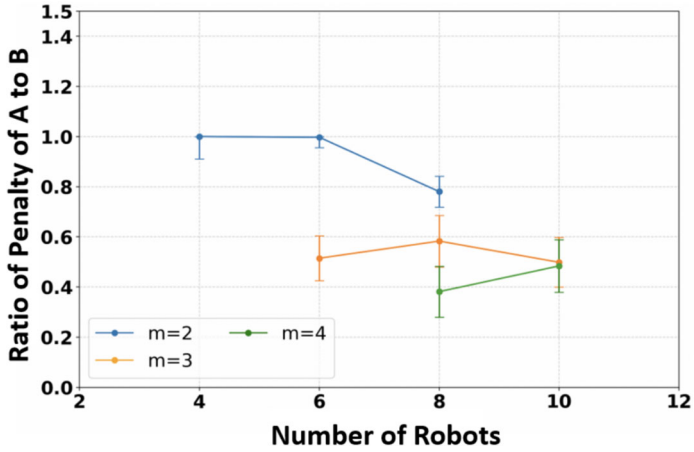

Fig. 2 Ratio of penalty where **A** is the optimal and **B** is the proposed approach: **CASE-2**

5.1 Simulation Setup

We have implemented the proposed algorithms in Python 3. 11.3 on Ubuntu 20.04 LTS, and conducted simulation experiments on a PC equipped with a 4.2 GHz Intel NUC (NUC10i5FNHN) CPU and 8 GB RAM. We experimented with task counts m ranging from 2 to 400 and robot counts n between 4 and 2000, ensuring that the number of tasks never exceeded 50% of the available robots in any test case. The unique 2D positions of robots and tasks were randomly assigned within a confined square grid environment measuring $100m \times 100m$. Additionally, the required skill set S was varied from RS_2 to RS_{100}, with skill distributions randomized across both robots and tasks. We have varied the speed of robots (v), and the duration to execute the task (δ) from the set {1, 10}, and {40, 100}, respectively. Each configuration was executed 50 times, and the average results are presented next. The bars in the plots represent each metric's maximum and minimum values.

5.2 Scenarios

To assess the quality of our proposed approach, we performed simulations across various scenarios using four distinct methods. These scenarios featured a range of task deadlines. For consistency, we employed the same deadline assignment strategies and data generation techniques for real-time scheduling as outlined in [33]. Table 1 shown the possible applications of the different deadline configurations. CF-HMRTA primarily focused on variations in task deadlines. The simulations considered the following deadline configurations D:

- **CASE-1:** $D = E$ (execution time – the distance between the task and the nearest robot capable of performing at least one required skill of the task divided by the minimum velocity of that robot) - In this scenario, configuring the deadline to match the execution time can result in a significant number of tasks missing their deadlines when there is a large number of robots. Although such

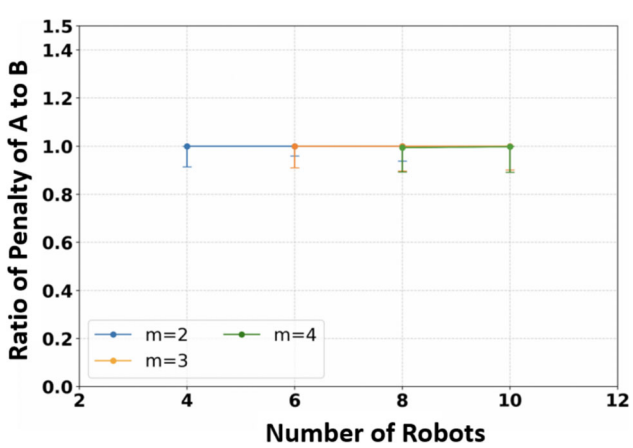

Fig. 3 Ratio of penalty where **A** is the optimal and **B** is the proposed approach: **CASE-3**

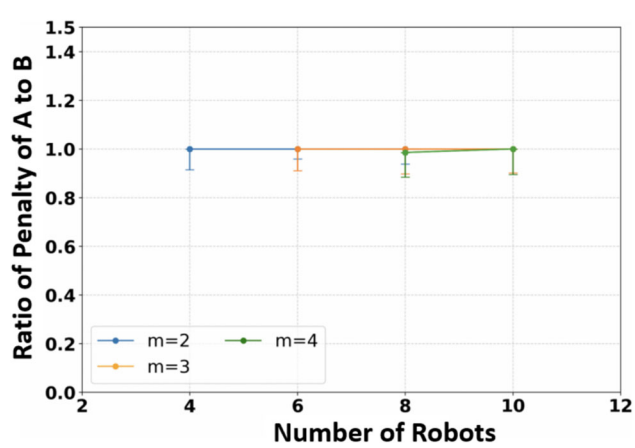

Fig. 4 Ratio of penalty where **A** is the optimal and **B** is the proposed approach: **CASE-4**

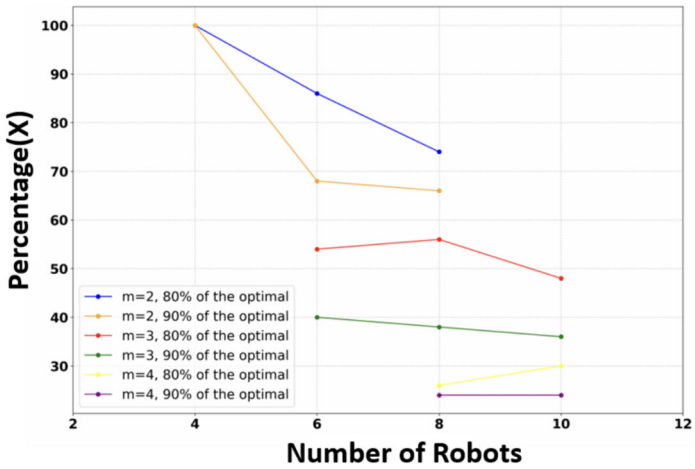

Fig. 5 **X** is equal to tests in which CF- HMRTA produced a coalition structure with effectiveness levels of 80% and 90% of the optimal: **CASE-1**

a situation is unlikely to occur frequently in real-world environments, it provides valuable insight into the scheduler's performance under worst-case conditions.

- **CASE-2:** $D = 2E$ - In this configuration, the deadline is set to twice the execution time of the task. While such a scenario is uncommon in practical environments, some tasks may still miss their deadlines when a higher number of robots is required. This challenging setup offers an additional perspective on the scheduler's performance.
- **CASE-3:** $D \in [5E, 10E]$ - In this case, the deadline is set using a uniform distribution in the range of $5E$ and $10E$. This approach enables a higher number of tasks to be completed within their deadlines.
- **CASE-4:** $D \in [E, 10E]$ - In this scenario, the deadline is selected randomly with equal probability from the previous three configurations. This approach prevents the scheduler from favoring any specific deadline setup and provides a more balanced performance evaluation.

5.3 Results and Discussions

Following the setup detailed earlier, we now present and discuss the results.

5.3.1 Solution Quality

At the outset, our main goal is to investigate the ratio of penalty incurred in CF- HMRTA to that incurred by the optimal solution as a performance indicator. A higher value of the ratio signifies better performance, with the value of 1 representing the optimal solution attained by the proposed method. We have used the brute-force method described in [34] to find the optimal solution. Subsequently, we calculate the ratio between the penalty incurred by our solution (CF-HMRTA) and that incurred by the optimal solution. The results for the worst-case scenario, where the deadline equals the execution time, are illustrated in Fig. 1, which shows the near-optimal

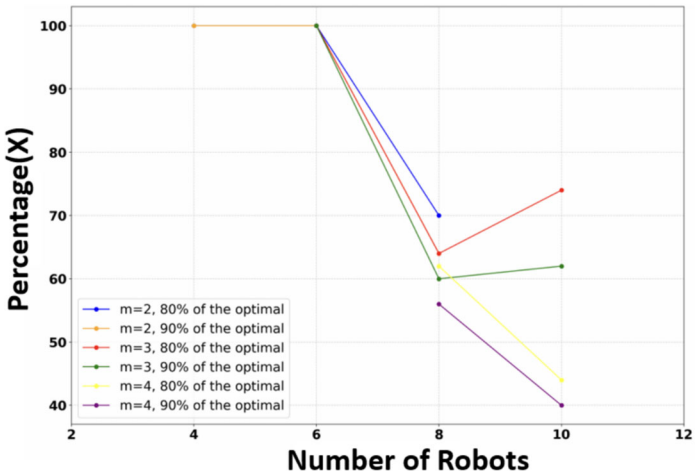

Fig. 6 **X** is equal to tests in which CF- HMRTA produced a coalition structure with effectiveness levels of 80% and 90% of the optimal: **CASE-2**

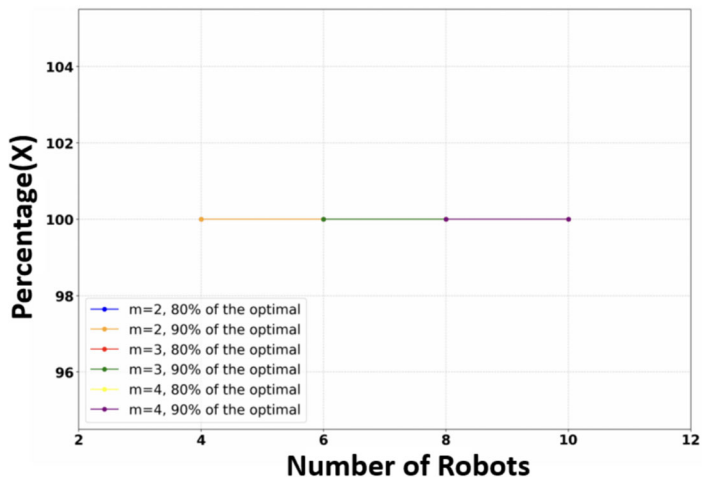

Fig. 7 **X** is equal to tests in which CF- HMRTA produced a coalition structure with effectiveness levels of 80% and 90% of the optimal: **CASE-3**

performance of our approach. To further assess the quality of our solution, we executed the brute-force algorithm for scenarios with $m = 2$ to $m = 4$ and $n = 4$ to $n = 10$, comparing our results against the optimal for the same parameters. For $m = 2$, the proposed CF- HMRTA method achieved values close to 90% of the optimal solution, with a maximum of 100% at $n = 4$ and a minimum of 75% at $n = 8$. As the number of tasks increases, the percentage of optimality decreases slightly, yet it remains comparable to the optimal solution. For $m = 4$, the optimality percentage ranges from a maximum of 72% at $n = 8$ to a minimum of 68.5% at $n = 10$. Due to the computationally intensive nature of the brute-force method, the test scenarios were limited to $m \in [2, 4]$ and $n \in [4, 10]$. When the deadline is increased to twice the execution time or more, the proposed method demonstrates improved stability, as depicted in Fig. 2. Furthermore, for deadlines ranging from in between five to ten times the execution time, as shown in Fig. 3, our approach consistently achieves 100% optimality for all robot and task configurations. In scenarios where deadlines vary between the execution time and ten times the execution time, as illustrated in Fig. 4, the proposed approach remains nearly 100% optimal across all configurations of robots and tasks.

Following the performance metrics used in [7], we also examine how frequently the algorithm attains specific thresholds of the optimal solution. In Fig. 5, for $m = 2$, the solution reaches at least 80% of the optimal 76% of time and at least 90% of the optimal approximately 67% of time over fifty simulation runs (for all n). For $m = 4$, the algorithm achieves 80% of the optimal in half the simulations (for all n), and in some instances–such as $n = 8$ and $n = 10$–this percentage increases slightly. Additionally, 90% of the optimal is

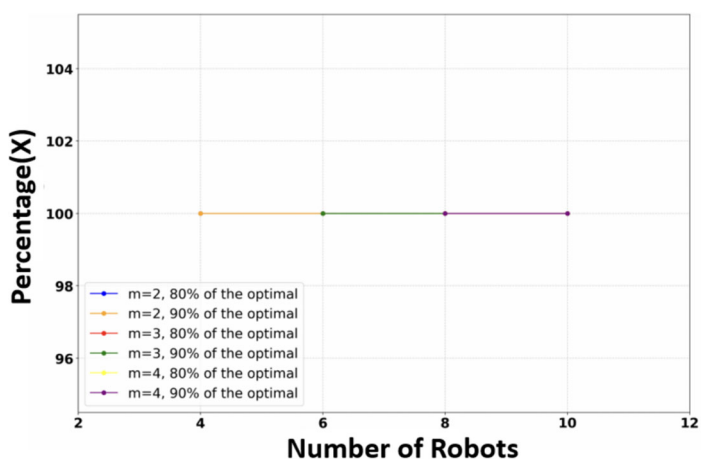

Fig. 8 **X** is equal to tests in which CF- HMRTA produced a coalition structure with effectiveness levels of 80% and 90% of the optimal: **CASE-4**

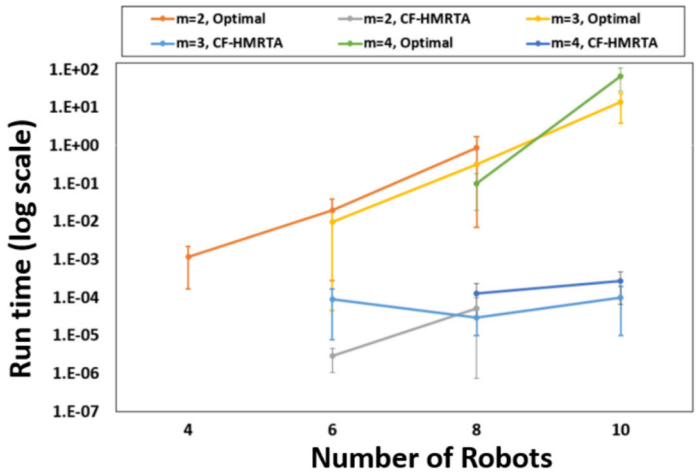

Fig. 9 Relation between computation time (in log scale) to a brute-force method [34]: **CASE-1**

reached half the time (for all n). When the deadline is doubled, performance improves significantly, as shown in Fig. 6. For $m = 2$, the solution attains at least 80% of the optimal in 94% of time and at least 90% of the optimal in about 70% of time over fifty simulations (for all n). For $m = 4$, the algorithm achieves 80% of the optimal in 44% of time and 90% of the optimal in 40% of time (for all n), as shown in Fig. 6. Furthermore, we evaluated scenarios where deadlines range from five to ten times the execution time. As the deadline increases, the solution consistently reaches 80% and 90% of the optimal 100% of the time, as shown in Fig. 7. Similarly, when deadlines vary from the execution time to ten times the execution time, the solution maintains the same performance, as illustrated in Fig. 8. This demonstrates the consistency of the presented task allocation technique.

5.3.2 Runtime

The computation time comparison between our CF- HMRTA approach and the brute-force solution is presented in Fig. 9 for scenarios where the deadline equals the execution time. This comparison highlights the computational efficiency of our method relative to the exhaustive brute-force approach. Analyzing runtime performance across varying numbers of tasks (m), a clear trend emerges: as m increases from 2 to 4, the runtime for our method rises from an impressively low $10.04\mu s$ to 0.12 ms. This increase is anticipated since a larger number of tasks naturally requires more complex computations, leading to longer processing times. Furthermore, Figs. 10 to 12 illustrate computation times under different deadline configurations, as discussed in Section 5.2.

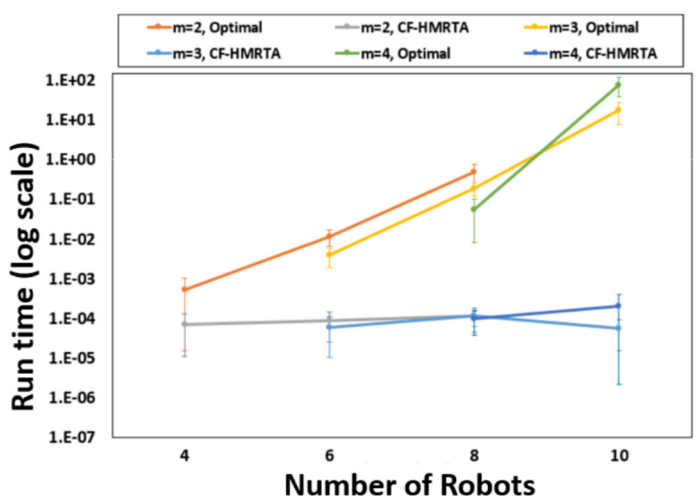

Fig. 10 Relation between computation time (in log scale) to a brute-force method [34]: **CASE-2**

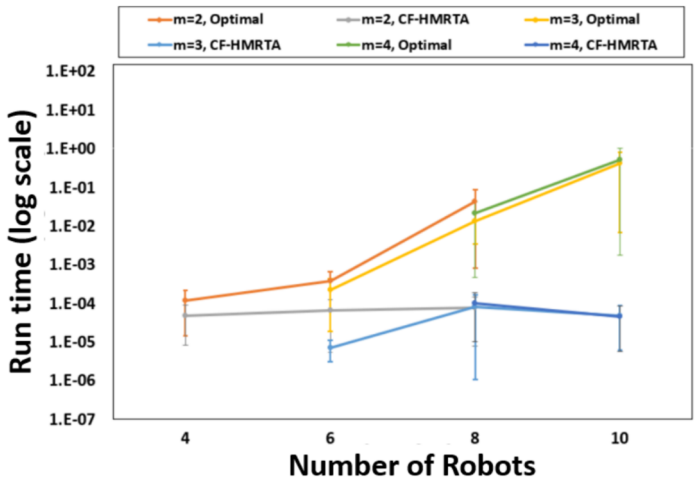

Fig. 11 Relation between computation time (in log scale) to a brute-force method [34]: **CASE-3**

When evaluating the brute-force method under the same conditions, we witness a significant surge in computation time. The computation time for the brute-force solution jumps from the same 2.3 ms to a significantly high value of 24 seconds on average. This illustrates a stark contrast in efficiency compared to our approach. Moreover, this discrepancy in computation time becomes even more pronounced as we further increase the value of m. The exponential nature of the brute-force approach [34] leads to an exponential increase in computation time as the number of tasks and robots grows. This highlights the scalability advantage of our proposed method.

5.3.3 Coalition Switching Analysis

In Fig. 13, it is evident that there are minimal switches for smaller values of m and n, as the average remains below 1. However, as m increases, particularly at $m = 4$, the number of switches rises significantly. For $n = 100$, the number of switches exceeds 18, with $m = 10$. The number of switches indicates the instability of the CF-NRMP approach, as not many robots find incentives to switch coalitions determined by Algorithm 2. This observation highlights a positive correlation between the number of switches and both the number of tasks and robots. Additionally, Algorithm 3 demonstrates

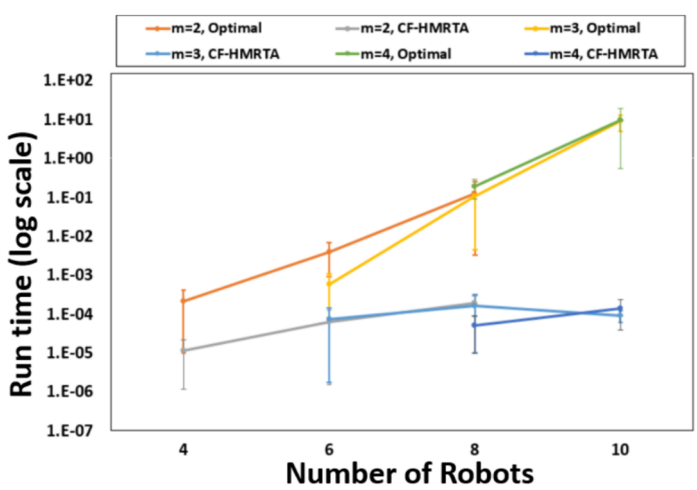

Fig. 12 Relation between computation time (in log scale) to a brute-force method [34]: **CASE-4**

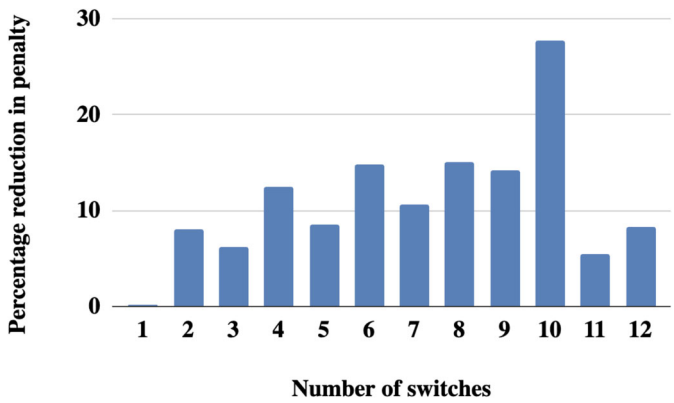

Fig. 13 Average number of switches performed

greater stability compared to Algorithm 2, as switching 10 robots reduces the overall penalty by 28%. This reduction becomes even more substantial as the number of robots increases, given that the overall penalty is on the order of $10^3 s$.

As previously mentioned, our aim in this research was to create an algorithm capable of finding near-optimal solutions at scale. Figures 14 and 15 illustrate the scalability of our proposed solution, with an average computation time of 4.7ms observed for $n = 100$ and $m = 10$. This computation time is negligible, particularly when considering the astronomical number of possible coalition structures for this setting, which is 2.75×10^{93} [7]. In our study, the robots aim to minimize the total incurred penalty for executing tasks Eq. 3, making it a critical performance metric. As shown in Figs. 16 to 17 for two different deadlines, the total incurred penalty by the robots increases sub-linearly with the number of robots, regardless of the task count. Furthermore, the average penalty of the CF- HMRTA is lower than that of CF-NRMP as the number of tasks and robots increases. This occurs because the number of switches rises with an increase in the robot count. This result demonstrates that the solution quality, in terms of the penalty metric, does not degrade significantly as the task and robot count increases.

5.3.4 Scalability Evaluations

To assess the scalability of our proposed approach in large-scale multi-robot systems, we perform an empirical analysis using the same total number of tasks and robots as in [29]. The findings, illustrated in Figs. 18 to 19, indicate a linear increase in the average penalty incurred by robots to reach their assigned task locations as n grows, consistent with our previous experiments involving up to 100 robots. Notably,

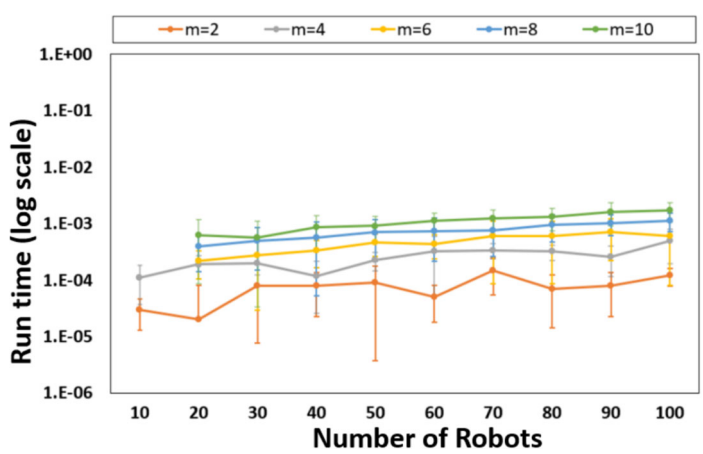

Fig. 14 Computation time (in log scale) of the proposed approach for different values of m and n: **CASE-1**

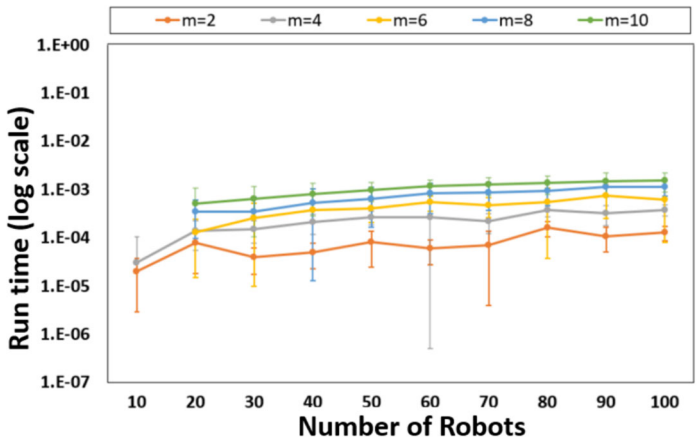

Fig. 15 Computation time (in log scale) of the proposed approach for different values of m and n: **CASE-2**

the runtime for 2000 robots and 400 tasks is just 11 seconds (Figs. 20 and 21), compared to approximately 50 seconds required by the approach in [32] for the same configuration (albeit in a different hardware setup).

Scaled-Up Robot and Task Configurations In Figs. 18 to 19, we examine the relationship between the number of deployed robots and the total penalty incurred across multiple runs. The results exhibit a clear upward trend, indicating that the total penalty increases sub-linearly as the number of robots grows. This consistent behavior across various configurations underscores the critical role of efficient coalition formation in minimizing penalties.

Coalition Switching and Its Implications Figures22 and 23 illustrate the number of robot switches for $m = 25$ tasks and $n = 2000$ robots under two deadline scenarios: $D = E$ and $2E$, respectively. For deadlines $D \in [5E, 10E]$ and $D \in [E, 10E]$, all tasks can be scheduled with a minimal penalty, and therefore, the corresponding results are omitted. The results reveal that approximately 300 robots switch from the CF-NRMP coalition to CF- HMRTA in these scenarios. This switching leads to a substantial 20% reduction in the total penalty, which is particularly significant given that the penalty is on the order of 10^3. Further testing with the same configuration ($m = 400, n = 2000$) highlights that nearly one-third of the robots transition from the CF-NRMP

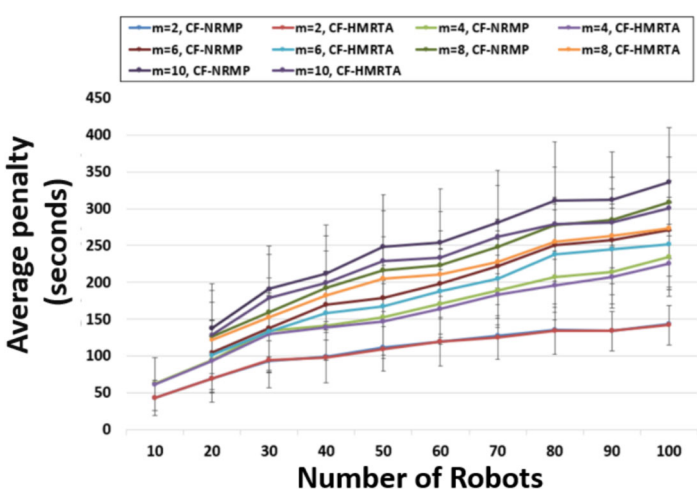

Fig. 16 The total penalty incurred (in seconds) by the robots to complete the assigned tasks for $m = 10$ and $n = 100$: **CASE-1**

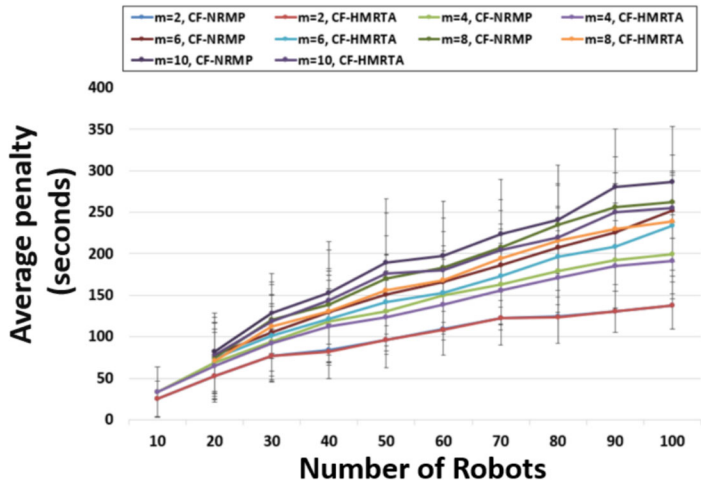

Fig. 17 The total penalty incurred (in seconds) by the robots to complete the assigned tasks for $m = 10$ and $n = 100$: **CASE-2**

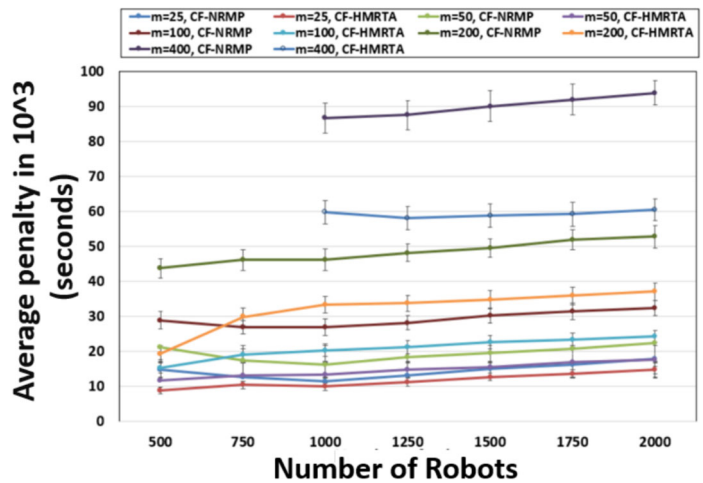

Fig. 18 The total penalty incurred (in seconds) by the robots to complete the assigned tasks for $m = 400$ and $n = 2000$: **CASE-1**

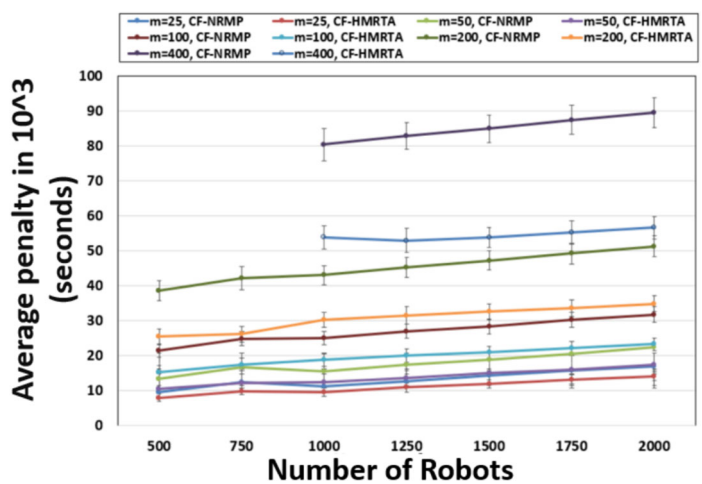

Fig. 19 The total penalty incurred (in seconds) by the robots to complete the assigned tasks for $m = 400$ and $n = 2000$: **CASE-2**

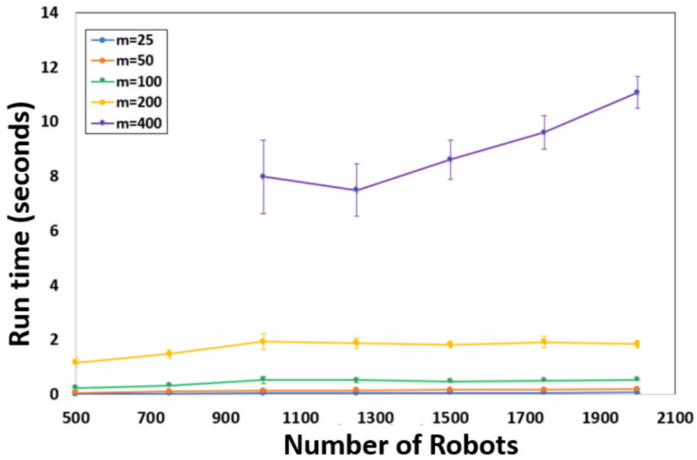

Fig. 20 Computation time (in log scale) of the proposed approach for different values of m and n: **CASE-1**

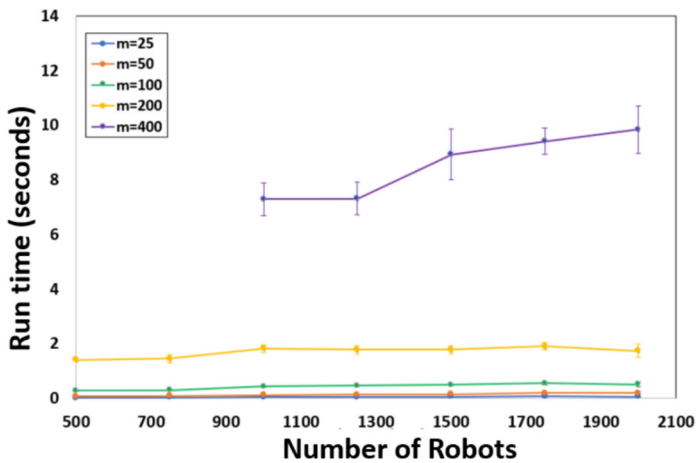

Fig. 21 Computation time (in log scale) of the proposed approach for different values of m and n: **CASE-2**

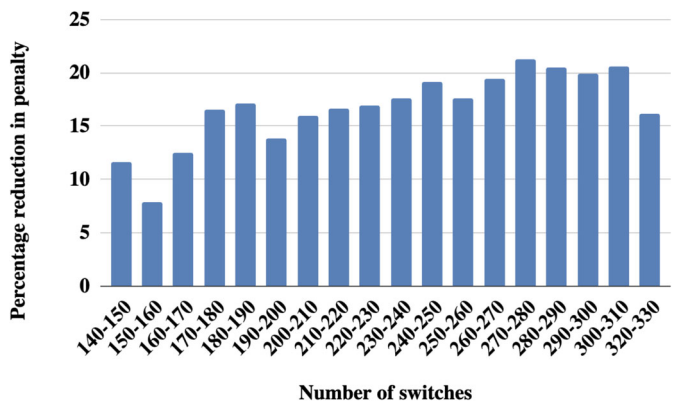

Fig. 22 Average number of switches performed for $m = 25$ and $n = 2000$: **CASE-1**

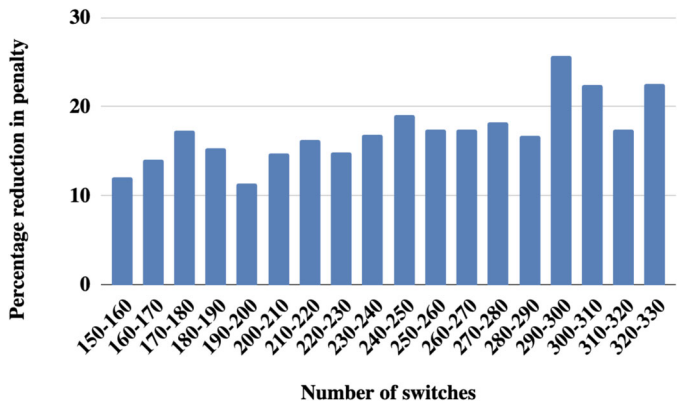

Fig. 23 Average number of switches performed for $m = 25$ and $n = 2000$: **CASE-2**

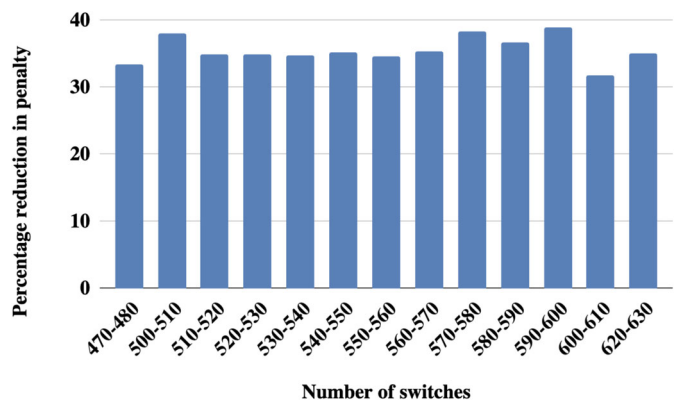

Fig. 24 Average number of switches performed for $m = 400$ and $n = 2000$: **CASE-1**

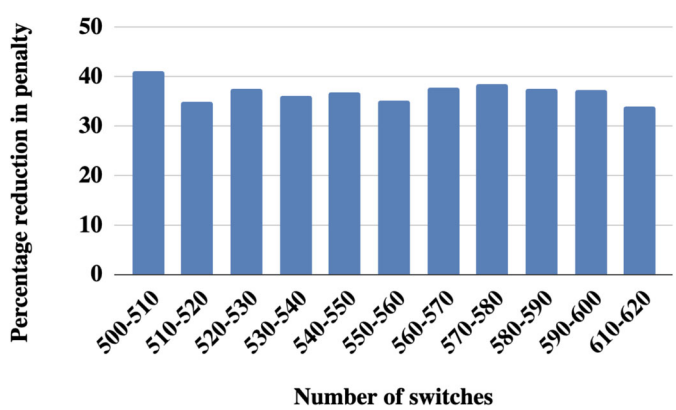

Fig. 25 Average number of switches performed for $m = 400$ and $n = 2000$: **CASE-2**

coalition to CF- HMRTA , as shown in Figs. 24 and 25, for deadlines $D \in [5E, 10E]$ and $D \in [E, 10E]$, respectively. This high frequency of switching underscores the increased stability and adaptability of the proposed approach compared to CF-NRMP. Notably, the penalty is reduced by approximately 37%, a considerable improvement that makes the proposed approach not only more stable but also more optimal in minimizing penalties. The results presented in Figs. 22 to 25 clearly demonstrate the effectiveness of the CF- HMRTA framework. By facilitating efficient coalition formation, the proposed approach significantly enhances system performance, reduces penalties, and ensures stability even in large-scale configurations with demanding deadline constraints.

6 Conclusions

This paper presents a novel coalition formation technique among heterogeneous robots for task allocation, which has real-world implications in applications such as warehouse automation and battlefield surveillance, among others. Prior research has established the NP-Hard nature of this problem, posing challenges in finding optimal solutions. In this paper, we propose an approach, named CF- HMRTA , inspired by the National Resident Matching Program, leveraging graph-matching principles for coalition formation. We have empirically demonstrated that our proposed solution is fast, and almost always finds near-optimal solutions. Furthermore, CF- HMRTA could easily scale to large multi-robot systems in our experiments. This is also underpinned by the worst-case time complexity analysis, which is proved to be a linear time-bound on the number of edges in the underlying bipartite graph.

The CF-HMRTA framework operates under the assumptions of centralized coordination, static task sets, and deterministic task execution. Moreover, its evaluation in this study has been limited to comparisons with brute-force and CF-NRMP approaches. To overcome these limitations, future work will focus on developing a distributed variant of CF-HMRTA that removes the reliance on a central planner, thereby eliminating the single point of failure. This extended framework will also incorporate task prioritization mechanisms to better handle varying task urgencies. The proposed distributed algorithm will be rigorously benchmarked against a broader range of state-of-the-art distributed coalition formation methods, with the goal of demonstrating its scalability, robustness, and effectiveness in more dynamic and realistic environments. Moreover, we will also consider the geographical distribution of tasks, such as clustered versus scattered arrangements, to evaluate the influence of spatial patterns on task allocation performance. These enhancements aim to significantly improve the framework's practical applicability while preserving its computational efficiency.

Acknowledgements The authors would like to acknowledge the students of INSPIRE-LAB, Department of Computer Science, BITS Pilani, for helping with the experiments. We would also like to thank the Department of Computer Science, BITS Pilani, for providing the necessary resources and support.

Author Contributions All authors contributed to the study's conception and design. The first draft of the manuscript was written by Ashish Verma, and all authors reviewed and commented on previous versions of the manuscript. All authors read and approved the final manuscript.

Funding This research received no specific grant from any funding agency.

Data Availability The dataset used in this paper was self-generated for research purposes and is available on GitHub https://github.com/Verma889Ashish/CF-HMRTA along with the code.

Declarations

Competing interests The authors have no competing interests to declare that are relevant to the content of this article.

Open Access This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit http://creativecommons.org/licenses/by/4.0/.

References

1. Phillips, A.E., Waterer, H., Ehrgott, M., Ryan, D.M.: Integer programming methods for large-scale practical classroom assignment problems. Comput. Oper. Res. **53**, 42–53 (2015)
2. Varone, S., Beffa, C.: Dataset on a problem of assigning activities to children, with various optimization constraints. Data Brief **25**, 104168 (2019)
3. Karamanis, R., Anastasiadis, E., Angeloudis, P., Stettler, M.: Assignment and pricing of shared rides in ride-sourcing using combinatorial double auctions. IEEE Trans. Intell. Trans. Syst. **22**(9), 5648–5659 (2021). https://doi.org/10.1109/TITS.2020.2988356
4. Long, C., Wong, R.C.-W., Peng, Y., Ye, L.: On good and fair paper-reviewer assignment. In: 2013 IEEE 13th International Conference on Data Mining, pp. 1145–1150 (2013). https://doi.org/10.1109/ICDM.2013.13
5. Gerkey, B.P., Matarić, M.J.: A formal analysis and taxonomy of task allocation in multi-robot systems. Int. J. Robot. Res. **23**(9), 939–954 (2004). https://doi.org/10.1177/0278364904045564

6. Ferreira, B.A., Petrović, T., Bogdan, S.: Distributed mission planning of complex tasks for heterogeneous multi-robot teams. arXiv:2109.10106. (2021). https://doi.org/10.48550/arXiv.2109.10106
7. Dutta, A., Asaithambi, A.: One-to-many bipartite matching based coalition formation for multi-robot task allocation. In: 2019 International Conference on Robotics and Automation (ICRA), pp. 2181–2187 (2019). https://doi.org/10.1109/ICRA.2019.8793855. IEEE
8. Korsah, G.A., Stentz, A., Dias, M.B.: A comprehensive taxonomy for multi-robot task allocation. Int. J. Robot. Res. **32**(12), 1495–1512 (2013). https://doi.org/10.1177/0278364913496484
9. Czarnecki, E., Dutta, A.: Scalable hedonic coalition formation for task allocation with heterogeneous robots. Intell. Serv. Robot. **14**(3), 501–517 (2021). https://doi.org/10.1007/s11370-021-00372-9
10. Ackerman Viden, O., Trabelsi, Y., Xu, P., Sankararaman, K.A., Maksimov, O., Kraus, S.: Allocation problem in remote teleoperation: Online matching with offline reusable resources and delayed assignments. In: Proceedings of the 2023 International Conference on Autonomous Agents and Multiagent Systems, pp. 513–521 (2023)
11. Roth, A.E., Peranson, E.: The effects of the change in the nrmp matching algorithm. JAMA **278**(9), 729–732 (1997). https://doi.org/10.1001/jama.1997.03550090053032
12. Soni, A., Dasannacharya, C., Gautam, A., Shekhawat, V.S., Mohan, S.: Multi-robot unknown area exploration using frontier trees. In: 2022 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), pp. 9934–9941 (2022). https://doi.org/10.1109/IROS47612.2022.9981914
13. Chevaleyre, Y., Dunne, P.E., Endriss, U., Lang, J., Lemaitre, M., Maudet, N., Padget, J., Phelps, S., Rodrígues-Aguilar, J.A., Sousa, P.: Issues in multiagent resource allocation. (2005)
14. Gautam, A., Mohan, S.: State: Distributed algorithm for uniform circle formation by multiple mobile robots. Intell. Serv. Robot. **9**, 347–366 (2016). https://doi.org/10.1007/s11370-016-0205-6
15. Dolgov, D.A., Durfee, E.H.: Resource allocation among agents with mdp-induced preferences. J. Artif. Intell. Res. **27**, 505–549 (2006). https://doi.org/10.1613/jair.2102
16. Gautam, A., Mohan, S.: A review of research in multi-robot systems. In: 2012 IEEE 7th International Conference on Industrial and Information Systems (ICIIS), pp. 1–5 (2012). https://doi.org/10.1109/ICIInfS.2012.6304778
17. Shehory, O., Kraus, S.: Methods for task allocation via agent coalition formation. Artif. Intell. **101**(1–2), 165–200 (1998). https://doi.org/10.1016/S0004-3702(98)00045-9
18. Service, T.C., Adams, J.A.: Coalition formation for task allocation: Theory and algorithms. Auton. Agents Multi-Agent Syst. **22**(2), 225–248 (2011)
19. Lau, H.C., Zhang, L.: Task allocation via multi-agent coalition formation: taxonomy, algorithms and complexity. In: Proceedings. 15th IEEE International Conference on Tools with Artificial Intelligence, pp. 346–350 (2003). https://doi.org/10.1109/TAI.2003.1250210
20. Manne, F., Bisseling, R.H.: A parallel approximation algorithm for the weighted maximum matching problem. In: International Conference on Parallel Processing and Applied Mathematics, pp. 708–717 (2007). https://doi.org/10.1007/978-3-540-68111-3_74. Springer
21. Munkres, J.: Algorithms for the assignment and transportation problems. J. Soc. Ind. Appl. Math. **5**(1), 32–38 (1957). https://doi.org/10.1007/978-3-540-68111-3_74
22. Liu, L., Shell, D.A.: Physically routing robots in a multi-robot network: Flexibility through a three-dimensional matching graph. Int. J. Robot. Res. **32**(12), 1475–1494 (2013). https://doi.org/10.1177/0278364913498788
23. Zhang, Y., Parker, L.E.: Considering inter-task resource constraints in task allocation. Auton. Agents Multi-Agent Syst. **26**, 389–419 (2013)
24. Chen, X., Zhang, P., Du, G., Li, F.: A distributed method for dynamic multi-robot task allocation problems with critical time constraints. Robot. Auton. Syst. **118**, 31–46 (2019)
25. Ghassemi, P., Chowdhury, S.: Multi-robot task allocation in disaster response: Addressing dynamic tasks with deadlines and robots with range and payload constraints. Robot. Auton. Syst. **147**, 103905 (2022)
26. Guerrero, J., Oliver, G., Valero, O.: Multi-robot coalitions formation with deadlines: Complexity analysis and solutions. PloS One **12**(1), 0170659 (2017)
27. Luo, L., Chakraborty, N., Sycara, K.: Distributed algorithm design for multi-robot task assignment with deadlines for tasks. In: 2013 IEEE International Conference on Robotics and Automation, pp. 3007–3013 (2013). IEEE
28. Diehl, G., Adams, J.A.: Grape-s: Near real-time coalition formation for multiple service collectives. arXiv:2310.12480. (2023)
29. Sarkar, C., Agarwal, M.: Cannot avoid penalty? let's minimize. In: 2019 International Conference on Robotics and Automation (ICRA), pp. 1052–1058 (2019). https://doi.org/10.1109/ICRA.2019.8794338
30. Song, T., Tong, Y., Wang, L., She, J., Yao, B., Chen, L., Xu, K.: Trichromatic online matching in real-time spatial crowdsourcing. In: 2017 IEEE 33rd International Conference on Data Engineering (ICDE), pp. 1009–1020 (2017). https://doi.org/10.1109/ICDE.2017.147
31. Williams, K.J.: A reexamination of the nrmp matching algorithm. National resident matching program. Academic medicine: journal of the Association of American Medical Colleges. **70**(6), 470–6 (1995). https://doi.org/10.1097/00001888-199506000-00007
32. Sarkar, C., Dey, S., Agarwal, M.: Semantic knowledge driven utility calculation towards efficient multi-robot task allocation. In: 2018 IEEE 14th International Conference on Automation Science and Engineering (CASE), pp. 144–147 (2018). https://doi.org/10.1109/COASE.2018.8560555
33. Baker, T.P.: Comparison of empirical success rates of global vs. partitioned fixed-priority and edf scheduling for hard real time. Dept. of Computer Science, Florida State University, Tallahasee, Tech. Rep. TR-050601. (2005)
34. Orlov, M.: Efficient generation of set partitions. Engineering and Computer Sciences, University of Ulm, Tech. Rep (2002)

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Ashish Verma is a Ph.D. scholar at the Augmented and Virtual Reality Lab, where he focuses on multi-robot task allocation, dynamic scheduling, and intelligent coordination in heterogeneous autonomous systems. He has published several papers on genetic algorithms, reinforcement learning, and real-time delivery optimization. His work on HMR-ODTA has significantly contributed to improving logistics in time-sensitive environments like hospitals and smart warehouses. His research interests include multi-agent systems, AI-based task allocation, and robotics in healthcare logistics.

Avinash Gautam is an Associate Professor in the Department of Computer Science and Information Systems at BITS Pilani, Pilani Campus. He earned his Ph.D. in 2017 from BITS Pilani, specializing in Multi-Robot Systems. He has published papers in renowned journals and conferences including ICCV, IROS, CASE, SMC, Journal of Intelligent Robotic Systems, Intelligent Service Robotics, and Neural Computing and Applications.

In addition to his research contributions, he has actively engaged with the academic community as a reviewer for several publications, including IEEE Access, IEEE Transactions on Intelligent Vehicles, IEEE Transactions on Cybernetics, Journal of Intelligent and Robotic Systems, Intelligent Service Robotics, Transactions of the Canadian Society for Mechanical Engineering, and Robotica. He is a professional member of the IEEE, IEEE Robotics and Automation Society, and ACM.

Ayan Dutta received his Ph.D. degree from the Computer Science Department, University of Nebraska at Omaha, in 2017. He is currently an Assistant Professor with the School of Computing, University of North Florida. His current research interests include multi-robot systems, modular robotics, and AI-based methods for robotic planning.

Virendra Singh Shekhawat received his Ph.D. degree in Computer Science and Engineering from the Birla Institute of Technology and Science, Pilani, India, in 2014. He is currently an Associate Professor and also holds the position of Faculty In-charge for Educational Technology support.

His main research interests include computer networking, distributed systems, computer architecture, delay-tolerant networks (DTN), multi-robot coordination systems, and software-defined networks (SDN). He has published research papers in journals like the IUP Journal of Telecommunications, Journal on Communications, Network and System Sciences, and international conferences such as IROS and SMC. He is a professional member of the IEEE and ACM.

Sudeept Mohan received his Ph.D. degree in Control of Robot Manipulators from the Birla Institute of Technology and Science, Pilani, in 2007. He is now a Professor at the Birla Institute of Technology and Science, Pilani. He previously served as Head of the Department of Computer Science and Information Systems and as Dean of the Admissions division.

He has completed two sponsored research projects from the Centre for Artificial Intelligence and Robotics (CAIR), DRDO, India, as Principal Investigator. He has published research papers in reputed journals such as Intelligent Service Robotics, Journal of Intelligent Robotic Systems, International Journal of Embedded Systems, and Sustainable Computing: Informatics and Systems, and in international conferences including IROS, SMC, and ECRTS. His research interests include multi-agent-based robotic systems, robot manipulator control, and computer architecture.